Mathematical Foundations and Implementation of Area Minimization under a Volume Constraint

Droplet relaxation with wetting, a moving obstacle and heterogeneous substrates – Theory (Part I) and Implementation (Part II)

Companion document to the *Droplet Relaxer* project

October 21, 2026

Abstract

This document describes the `droplet_relaxer` program: the relaxation of a liquid droplet on a flat substrate – optionally pulled by a moving rigid sphere – to its minimal-energy shape at fixed volume, implemented as discrete gradient descent on a triangle mesh, at its core the same principle as Ken Brakke's *Surface Evolver*. **Part I (Theory)** is independent of the code: the continuous variational problem, the interfacial (wetting) energies including a circular wettability anomaly, the force on the sphere, the discretization with explicit derivations of all gradients, and the optimization theory (constraints, Sobolev gradients, step control). **Part II (Implementation)** describes how this is realized in the program: module structure, meshes and scaling, assembly of energies and forces in each iteration, the solver, remeshing, visualization and output, the purely numerical ingredients that have no physical origin, validation results and known limitations. **Part III** compares the project with the Surface Evolver and lists possible extensions.

Contents

1 Motivation and Overview **2**

I Theory **3**

2 The Continuous Variational Problem **3**
2.1 The Area Functional and Its First Variation . . . 3
2.2 The Volume Constraint . . . 3
2.3 Boundary Condition at the Contact Line . . . 4

3 Interfacial Energies and Wetting **4**
3.1 Young's Energy and the Wall Energy . . . 4
3.2 A Substrate with Spatially Varying Wettability . . . 5
3.3 Wetting Energy on a Rigid Sphere . . . 6
3.4 The Force on the Sphere . . . 6
3.5 Centre of Mass . . . 6

4 Discretization: The Surface as a Simplicial Complex **7**
4.1 Notation . . . 7
4.2 The Discrete Area Functional . . . 7
4.3 Derivation of the Area Gradient . . . 7
4.4 The Discrete Volume Functional . . . 8

4.5 Derivation of the Volume Gradient . . . 8
4.6 Connection to Discrete Mean Curvature . . . 8
4.7 Discrete Contact-Line Energies . . . 9
4.8 Discrete Centre of Mass . . . 11

5 Constraints and the Optimisation Method **11**
5.1 Exact Lagrange Projection . . . 11
5.2 Penalty Formulation . . . 11
5.3 Non-Penetration of a Rigid Sphere . . . 11
5.4 Gradient Flow and Explicit Euler Integration . . . 12
5.5 Sobolev Preconditioning . . . 12
5.6 The Discrete Laplace Operator . . . 13
5.7 Trust-Region Step Capping . . . 13

II Implementation **13**

6 Program Structure **13**

7 Meshes, Scaling and Degrees of Freedom **13**

8 Assembly of Energy and Force **15**
8.1 Wetted Set and Contact Loop . . . 15
8.2 Anomaly . . . 16
8.3 Force on the Sphere . . . 16

9 The Iteration: Step and Numerical Devices **16**

10 Adaptive Remeshing **17**
10.1 Two Refinement Modes . . . 17
10.2 Edge Collapse . . . 18
10.3 Triggers and Mesh-Quality Smoothing . . . 18
10.4 Cost and Speed-ups . . . 19

11 Output and Visualization **19**

12 Verification **19**

13 Mapping from Equations to Code **20**

III Comparison and Outlook **20**

14 Comparison to Ken Brakke's Surface Evolver **20**
14.1 Where the Two Agree . . . 20
14.2 Where This Project Is Narrower . . . 20

15 Context and Outlook **22**

1 Motivation and Overview

A liquid droplet on a substrate takes on the shape that minimizes its interfacial energy while its volume stays constant (liquid is incompressible). Since the liquid/vapor energy is proportional

to area, this leads to a classical *constrained variational problem*:

$$\min_{\Sigma} A[\Sigma] \quad \text{subject to} \quad V[\Sigma] = V_0, \tag{1}$$

where Σ is a surface with a boundary curve (the contact line) on the substrate, $A[\Sigma]$ is its area, and $V[\Sigma]$ is the enclosed volume. Wetting adds energies on the contact area with the substrate and with a rigid obstacle (Section 3).

Kenneth Brakke's Surface Evolver [2] solves exactly this problem (and substantially more general variants of it) numerically, by discretizing the surface as a triangle mesh and iteratively moving the vertices along the negative gradient of the energy – a discrete gradient descent. Our program mirrors the same principle, with two improvements over the simplest approach: a Sobolev preconditioning of the gradient (Section 5.5) and a penalty formulation of the volume constraint (Section 5).

How this document is organized. *Part I* states the model and derives the mathematics; it contains no program names or parameter values and would be valid for any implementation. *Part II* states what the program does with it: which quantities are computed where, which numerical devices were added that are *not* implied by the theory, how the output is produced and how it was checked. Cross-references between the parts point from each implementation item to the equation it realizes (Table 2 collects them).

Part I
Theory

2 The Continuous Variational Problem

2.1 The Area Functional and Its First Variation

Let $\Sigma \subset \mathbb{R}^3$ be a smooth, oriented surface with unit normal n. Consider a family of surfaces Σ_t obtained by displacing every point along the normal:

$$x(t) = x_0 + t\,\varphi(x_0)\,n(x_0), \qquad x_0 \in \Sigma, \tag{2}$$

with a scalar "speed function" φ. The first variation of the area functional $A[\Sigma_t] = \int_{\Sigma_t} \mathrm{d}A$ under this perturbation is a classical result of differential geometry:

$$\left.\frac{\mathrm{d}}{\mathrm{d}t}\right|_{t=0} A[\Sigma_t] = -\int_{\Sigma} H\,\varphi\,\mathrm{d}A, \tag{3}$$

where $H = \kappa_1 + \kappa_2$ is the mean curvature (here defined as the sum of the principal curvatures). Equation (3) states that the L^2-gradient of the area functional with respect to normal displacements is

$$\nabla A \;=\; -H\,n. \tag{4}$$

The steepest descent $\partial x/\partial t = -\nabla A = Hn$ is thus exactly the *mean curvature flow* – precisely the process that Brakke described mathematically, rigorously, within the framework of geometric measure theory, in his dissertation [1] (from which the Surface Evolver later grew).

2.2 The Volume Constraint

Displacing the surface by $\varphi\,n$ changes the enclosed volume, to first order, by

$$\left.\frac{\mathrm{d}}{\mathrm{d}t}\right|_{t=0} V[\Sigma_t] = \int_{\Sigma} \varphi\,\mathrm{d}A, \qquad \text{i.e.} \qquad \nabla V = n. \tag{5}$$

For the minimization problem (1), the Lagrange multiplier method gives: at a critical point there exists $\lambda \in \mathbb{R}$ such that

$$\nabla A - \lambda \nabla V = 0 \quad \Longleftrightarrow \quad H = \lambda = \text{const.} \tag{6}$$

This is the classical theorem that area-minimizing surfaces under a fixed volume constraint have *constant mean curvature* (spherical caps, soap bubbles) – the Lagrange multiplier λ is physically the internal pressure (the Young–Laplace law).

2.3 Boundary Condition at the Contact Line

At the boundary $\partial\Sigma$ (the contact line), there are two standard cases:

- **Pinned boundary (Dirichlet):** $\partial\Sigma$ is fixed in advance, e.g. a circle on the substrate.
- **Free contact angle (Neumann-type):** $\partial\Sigma$ is allowed to move freely on the substrate, but there is a substrate energy $(\gamma_{sl} - \gamma_{sv}) \cdot$ (wetted area). The first variation then leads to Young's equation $\cos\theta = (\gamma_{sv} - \gamma_{sl})/\gamma_{lv}$ for the equilibrium contact angle θ.

In the model treated here the boundary condition is a mixture of the two: the contact line is Dirichlet-constrained only in the direction *normal to the substrate* (it must stay on the wall, $z = 0$) but is otherwise free to slide within the substrate plane, with the free-contact-angle substrate energy added explicitly (Section 3.1). Without that energy term the boundary reduces to a pure natural (Neumann) condition and settles at $\theta = 90°$; the contact line is never pinned (fixed radius) in this model. The discrete degree-of-freedom restriction is described in Section 7.

3 Interfacial Energies and Wetting

This section states the physical energies that go beyond the pure area functional: the wall (liquid/solid) energy, a substrate whose wettability varies in space, the energy of the liquid in contact with a rigid sphere, and the force the liquid exerts on that sphere. All of it is formulated for smooth surfaces; the discrete counterparts follow in Section 4.7.

3.1 Young's Energy and the Wall Energy

Let the liquid be bounded by a liquid/vapor surface Σ_{lv} (area A_{lv}) and a wetted region of the substrate (liquid/solid interface, area A_{sl}). With the three interfacial tensions $\gamma_{lv}, \gamma_{sl}, \gamma_{sv}$ and a substrate of total area A_{wall} (a constant),

$$E = \gamma_{lv} A_{lv} + \gamma_{sl} A_{sl} + \gamma_{sv}(A_{\text{wall}} - A_{sl}) = \gamma_{lv} A_{lv} - (\gamma_{sv} - \gamma_{sl}) A_{sl} + \text{const.} \tag{7}$$

Moving the contact line outward by δ per unit length changes A_{sl} by $+\delta$ and, for a liquid/vapor surface meeting the wall at the interior angle θ, A_{lv} by $\cos\theta\,\delta$, so stationarity gives *Young's equation*

$$\gamma_{sv} - \gamma_{sl} = \gamma_{lv} \cos\theta_Y. \tag{8}$$

Writing $\kappa := \cos\theta_Y$, the total interface energy is therefore

$$E \;=\; \gamma_{lv} A_{lv} \;-\; \kappa\, \gamma_{lv}\, A_{sl}. \tag{9}$$

A positive κ (more wetting, $\theta_Y < 90°$) *rewards* a larger wetted area; $\kappa = 0$ ($\theta_Y = 90°$) means no wall energy at all. For the pure area problem of Section 2 we set $\gamma_{lv} = 1$: a constant factor does not change the minimizer. The wall part alone is

$$E_{\text{wall}} = -\kappa\, \gamma_{lv}\, A_{\text{proj}}, \tag{10}$$

where $A_{\mathrm{proj}} = A_{sl}$ is the area enclosed by the contact line's projection onto the (flat) substrate plane. By Green's theorem this area is a closed line integral over the contact line $\partial\Sigma$ – the planar analogue of the divergence-theorem trick used for the volume (Section 4.4):

$$A_{\mathrm{proj}} = \frac{1}{2}\oint_{\partial\Sigma}(x\,\mathrm{d}y - y\,\mathrm{d}x). \tag{11}$$

Consequently the wall energy needs no substrate surface mesh at all: it depends only on the contact line.

3.2 A Substrate with Spatially Varying Wettability

If the Young angle varies over the substrate, $\kappa = \kappa(x, y)$, the wall energy generalizes to an integral over the wetted region Ω_{wet} (the region of the plane enclosed by the contact line):

$$E_{\mathrm{wall}} = -\gamma_{lv}\int_{\Omega_{\mathrm{wet}}}\kappa(x, y)\,\mathrm{d}A. \tag{12}$$

Consider a homogeneous wall ($\kappa_0 = \cos\theta_0$) with a circular anomaly of centre $\mathbf{p}$, radius r and angle θ_a ($\kappa_a = \cos\theta_a$). With a smooth indicator $s(\rho)$ of the disc, $\rho = |\mathbf{x}-\mathbf{p}|$, $\kappa = \kappa_0 + (\kappa_a - \kappa_0)\,s$, and the energy splits into the homogeneous part (10) and

$$E_{\mathrm{anom}} = -\gamma_{lv}(\kappa_a - \kappa_0)\int_{\Omega_{\mathrm{wet}}} s(\rho)\,\mathrm{d}A. \tag{13}$$

The indicator is $s = 1$ for $\rho \le r - h$, $s = 0$ for $\rho \ge r + h$ and, in the transition zone of total width $2h$, the quintic "smootherstep"

$$s = 1 - (10u^3 - 15u^4 + 6u^5), \qquad u = \frac{\rho - (r-h)}{2h} \in [0, 1], \tag{14}$$

which is C^2 (so E_{anom} has a continuous second derivative with respect to the contact line). The width is parametrized as $2h = f\,r$ with a fraction f of the radius.

Reduction to a contact-line integral. Since s is radial, the area integral can again be written as a boundary integral. Let $S(\rho) = \int_0^\rho s(r')\,r'\,\mathrm{d}r'$ (a polynomial on the transition zone, closed form) and define the vector field

$$\mathbf{P}(\mathbf{x}) = \frac{S(\rho)}{\rho^2}\,(\mathbf{x} - \mathbf{p}). \tag{15}$$

Its divergence is $\operatorname{div}\mathbf{P} = \frac{1}{\rho}\frac{\mathrm{d}}{\mathrm{d}\rho}\big(\rho\cdot S/\rho\big) = S'(\rho)/\rho = s(\rho)$, hence by the divergence theorem

$$\int_{\Omega_{\mathrm{wet}}} s\,\mathrm{d}A = \oint_{\partial\Sigma}\mathbf{P}\cdot\mathbf{n}\,\mathrm{d}l, \tag{16}$$

with $\mathbf{n}$ the outward normal of the contact line in the plane. (At $\rho \to 0$, $\mathbf{P} \to (\mathbf{x} - \mathbf{p})/2$, so the field is regular.) The first variation of this boundary integral under a displacement $\delta\mathbf{x}$ of the contact line is

$$\delta\int_{\Omega_{\mathrm{wet}}} s\,\mathrm{d}A = \oint_{\partial\Sigma} s\,(\mathbf{n}\cdot\delta\mathbf{x})\,\mathrm{d}l, \tag{17}$$

that is, the local Young force $-\gamma_{lv}\kappa(\mathbf{x})\,\mathbf{n}$ at each point of the contact line, now with the local wettability $\kappa(\mathbf{x})$.

Consequences. (i) A patch lying completely *inside* the wetted region has no effect: only the part of the wetted region that the contact line sweeps over contributes to a change of the energy. (ii) For a hydrophilic patch ($\theta_a < \theta_0$) the energy as a function of the droplet position has a well of depth about $\gamma_{lv}(\kappa_a - \kappa_0)A_{\text{covered}}$, for a hydrophobic one a hill of that height. (iii) The model has no contact-angle hysteresis: pinning at the patch edge arises only from this energy barrier.

3.3 Wetting Energy on a Rigid Sphere

The same Young-type energy applies where the liquid touches a rigid sphere of radius R and centre $\mathbf{c}$: the wetted part of the sphere (liquid/solid interface, area A_{patch}) lowers or raises the energy,

$$E_{\text{wetting}} = -\kappa_{\text{sphere}}\,\gamma_{lv}\,A_{\text{patch}}, \qquad \kappa_{\text{sphere}} = \cos\theta_{Y,\text{sphere}}, \tag{18}$$

and the liquid/vapor area excludes the wetted facets. Unlike the plane, a sphere has no simple global vector potential for its area form, so the trick of Eq. (11) does not carry over directly – an arbitrarily oriented loop on a sphere is exactly the setting of the holonomy term in the Gauss–Bonnet theorem. The way out is to use Gauss–Bonnet itself: for a simple loop γ on the unit sphere bounding a region D, $\int_D K\,\mathrm{d}A = 2\pi - \oint_\gamma \kappa_g\,\mathrm{d}s$ with $K = 1$, i.e. the *solid angle* enclosed by the loop is

$$\Omega = 2\pi - \oint_\gamma \kappa_g\,\mathrm{d}s, \qquad A_{\text{patch}} = R^2\,\Omega, \tag{19}$$

which refers only to the loop itself (its geodesic curvature), not to any global potential or preferred direction. The price is that "the enclosed side" must be unambiguous, i.e. the loop must be small enough (it must not wrap around the sphere) and must be a single simple loop.

3.4 The Force on the Sphere

For a quasi-static process the force of the liquid on the sphere follows from virtual work, $\mathbf{F} = -\partial E/\partial \mathbf{c}$: the energy change per displacement of the sphere centre. The configuration of the liquid depends on $\mathbf{c}$ as well, but at equilibrium the liquid minimizes E for each $\mathbf{c}$, so by the envelope argument only the *explicit* dependence of E on $\mathbf{c}$ is needed, namely (i) the contact/collision terms and the Young term on the sphere patch, which depend on $\mathbf{v} - \mathbf{c}$, and (ii) the reaction of the constraint that holds the wetted liquid on the sphere surface, which carries the liquid pressure on the patch and the radial pull of the contact line. Out of equilibrium the same expression is the instantaneous force the current liquid configuration exerts on the sphere (force balance).

Symmetry. On a homogeneous substrate the whole system is invariant under lateral translations of sphere and droplet together, so the equilibrium energy $E^*(\mathbf{c})$ does not depend on the lateral position and $F_x = F_y = 0$ at equilibrium. A *lateral* force at equilibrium requires a symmetry-breaking substrate: with a wettability anomaly (Section 3.2) $E^*(X)$ has a well or hill and $F_x = -\mathrm{d}E^*/\mathrm{d}X$. What a moving sphere in a real liquid experiences as friction additionally contains viscous dissipation and contact-angle hysteresis, neither of which is in this energy model.

3.5 Centre of Mass

For a closed liquid surface the centroid of the enclosed (homogeneous) liquid is, by the divergence theorem applied componentwise,

$$\mathbf{r}_{\text{COM}} = \frac{1}{V}\int_{\text{liquid}} \mathbf{x}\,\mathrm{d}V, \tag{20}$$

which the discrete formula of Section 4.8 evaluates from the boundary alone. The offset of the centre of mass from the sphere centre in the substrate plane, $(d_x, d_y) = \mathbf{r}_{xy}^{\mathrm{COM}} - \mathbf{c}_{xy}$, $d = \sqrt{d_x^2 + d_y^2}$, is a convenient measure of how far the droplet lags behind or leads the sphere.

4 Discretization: The Surface as a Simplicial Complex

4.1 Notation

We replace the smooth surface Σ with a triangle mesh: a set of vertices $\{v_i\}_{i=1}^{N} \subset \mathbb{R}^3$ and a set of triangles (facets) T, where each triangle $t \in T$ is given by an ordered triple (i, j, k) of vertex indices. This piecewise-linear surface is exactly the *simplicial complex* that the Surface Evolver also uses as its data structure.

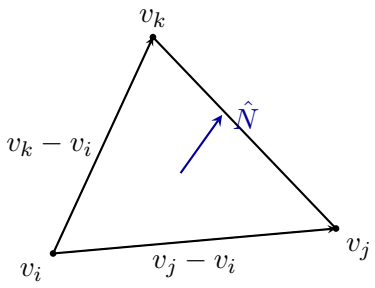


Figure 1: A triangle $t = (i, j, k)$ with edge vectors and unit normal $\hat{N}$. The triangle's area and its gradient with respect to each vertex are derived in Section 4.3.

4.2 The Discrete Area Functional

For a triangle $t = (i, j, k)$ we define the (unnormalized) normal vector

$$N_t \;=\; (v_j - v_i) \times (v_k - v_i), \qquad \hat{N}_t = \frac{N_t}{|N_t|}, \tag{21}$$

so that its area is $\mathrm{Area}_t = \frac{1}{2}|N_t|$. The discrete area functional is the sum over all triangles:

$$A(x) \;=\; \sum_{t \in T} \mathrm{Area}_t \;=\; \frac{1}{2} \sum_{t=(i,j,k)\in T} \big|(v_j - v_i) \times (v_k - v_i)\big|, \tag{22}$$

where $x = (v_1, \ldots, v_N) \in \mathbb{R}^{3N}$ is the vector of all vertex coordinates.

4.3 Derivation of the Area Gradient

We compute $\nabla_{v_i} \mathrm{Area}_t$ for a fixed triangle $t = (i, j, k)$. With $a := v_j$, $b := v_k$ held constant and $x := v_i$ the variable, we have $N_t(x) = (a - x) \times (b - x)$. The directional derivative in direction h is

$$\begin{aligned} \mathrm{d}N_t[h] &= (-h) \times (b - x) + (a - x) \times (-h) \\ &= (b - x) \times h - (a - x) \times h \\ &= \big[(b - x) - (a - x)\big] \times h \;=\; (b - a) \times h. \end{aligned} \tag{23}$$

Since $\mathrm{Area}_t = \frac{1}{2}|N_t|$ and $\mathrm{d}|N_t|[h] = \hat{N}_t \cdot \mathrm{d}N_t[h]$ (the derivative of a norm is the projection onto the unit direction), (23) gives

$$\mathrm{dArea}_t[h] = \tfrac{1}{2}\, \hat{N}_t \cdot \big[(b - a) \times h\big] = \tfrac{1}{2}\, h \cdot \big[\hat{N}_t \times (b - a)\big], \tag{24}$$

where we used the cyclic invariance of the scalar triple product $u \cdot (v \times w) = w \cdot (u \times v)$. Since the directional derivative is exactly $h \cdot \nabla_{v_i} \mathrm{Area}_t$, we can read off the gradient directly:

$$\boxed{\nabla_{v_i} \mathrm{Area}_t \;=\; \tfrac{1}{2}\, \hat{N}_t \times (v_k - v_j) \;=\; \tfrac{1}{2}\, (v_j - v_k) \times \hat{N}_t} \tag{25}$$

By cyclically permuting the roles of i, j, k (the triangle is symmetric under cyclic relabeling), we immediately obtain

$$\nabla_{v_j} \mathrm{Area}_t = \tfrac{1}{2}(v_k - v_i) \times \hat{N}_t, \qquad \nabla_{v_k} \mathrm{Area}_t = \tfrac{1}{2}(v_i - v_j) \times \hat{N}_t. \tag{26}$$

The total gradient of $A(x)$ with respect to a vertex v_i is the sum of (25)–(26) over all triangles containing v_i (the so-called "1-ring" around v_i).

4.4 The Discrete Volume Functional

For a closed surface, the divergence theorem with the vector field $F(x) = x/3$ (which satisfies $\operatorname{div} F \equiv 1$) gives

$$V = \int_{\text{volume}} 1 \, \mathrm{d}V = \int_{\Sigma} \tfrac{1}{3}\, x \cdot n \, \mathrm{d}A. \tag{27}$$

On a triangle $t = (i, j, k)$ (with matching orientation), this surface integral reduces to the well-known tetrahedron volume relative to the origin:

$$V_t = \tfrac{1}{6}\, v_i \cdot (v_j \times v_k), \tag{28}$$

and the total volume is the sum

$$V(x) = \frac{1}{6} \sum_{t=(i,j,k) \in T} v_i \cdot (v_j \times v_k). \tag{29}$$

For a *closed* surface, this value is independent of the choice of origin (the contributions of the "missing" opposite side cancel out telescopically). In our case (an open surface above a substrate plane $z = 0$ passing through the origin), $x \cdot n = 0$ holds for every point on the substrate – the "cap" contributes exactly zero to the sum (29), which is why we can compute the volume using only the free (upper) droplet surface, without explicitly triangulating the substrate cap.

4.5 Derivation of the Volume Gradient

Equation (29) is linear in each individual vertex (with the other two held fixed), so the derivative is immediate:

$$\frac{\partial}{\partial v_i} \big[v_i \cdot (v_j \times v_k) \big] = v_j \times v_k. \tag{30}$$

For the derivatives with respect to v_j and v_k, we again use the cyclic symmetry of the scalar triple product, $v_i \cdot (v_j \times v_k) = v_j \cdot (v_k \times v_i) = v_k \cdot (v_i \times v_j)$, and obtain

$$\boxed{\nabla_{v_i} V_t = \tfrac{1}{6}\, v_j \times v_k, \qquad \nabla_{v_j} V_t = \tfrac{1}{6}\, v_k \times v_i, \qquad \nabla_{v_k} V_t = \tfrac{1}{6}\, v_i \times v_j.} \tag{31}$$

4.6 Connection to Discrete Mean Curvature

A well-known result of discrete differential geometry [3] states that the area gradient at a vertex is proportional to the mean curvature vector:

$$\nabla_{v_i} A \;\approx\; 2\, H(v_i)\, n(v_i)\, A_{\text{mixed}}(v_i), \tag{32}$$

where $A_{\text{mixed}}(v_i)$ is a suitable "mixed" (Voronoi-like) area assigned to the vertex. This is consistent with (4): the continuous gradient $-Hn$ reappears discretely as a weighted mean curvature vector. We pragmatically approximate $A_{\text{mixed}}(v_i)$ as one third of the sum of the adjacent triangle areas (instead of the exact Voronoi cell) and obtain from it the *unsigned* curvature

$$H(v_i) \;\approx\; \frac{|\nabla_{v_i} A|}{2\, A_{\text{mixed}}(v_i)}, \tag{33}$$

which is used to colour the surface in the visualization – without having to implement a separate curvature operator.

A boundary caveat. Relation (32) assumes a *complete* triangle fan around v_i and is only exact for interior vertices. At a boundary vertex (a point on the contact line), the fan is incomplete, and the raw area gradient there picks up an additional contribution related to the geodesic curvature of the boundary curve itself – unrelated to the true surface curvature H. Numerically, this contribution can be an order of magnitude larger than the genuine curvature signal (in a typical run, boundary vertices show apparent curvatures around 60–70 versus a true local value near 0.5), which saturates the color scale and produces a spuriously bright rim in the visualization. The program corrects for this by replacing each boundary vertex's displayed curvature with the average of its interior neighbors' values; this is a purely cosmetic fix for the color map and does not affect the optimization itself, which never uses (32) directly (it works with the exact area/volume gradients of Section 4.3).

4.7 Discrete Contact-Line Energies

This subsection gives the discrete counterparts of the energies of Section 3. The contact line is the ordered closed polygon $p_1, \dots, p_n$ of dome vertices on the substrate (for the wall) or on the sphere (for the wetted patch).

Wall energy: shoelace formula. The wetted substrate area is the area of the planar polygon given by the (x, y) coordinates of the contact line,

$$A_{sl} = A_{\text{proj}} = \frac{1}{2} \sum_i \big(x_i y_{i+1} - x_{i+1} y_i \big), \tag{34}$$

(i cyclic; the planar analogue of Eq. (11)). Every other term of the sum is independent of vertex i except the two containing its neighbours, so

$$\frac{\partial A_{\text{proj}}}{\partial x_i} = \tfrac{1}{2}(y_{i+1} - y_{i-1}), \qquad \frac{\partial A_{\text{proj}}}{\partial y_i} = \tfrac{1}{2}(x_{i-1} - x_{i+1}), \tag{35}$$

and $\nabla_{p_i} E_{\text{wall}} = -\gamma_{lv} \cos\theta_Y \, (\partial_{x_i} A_{\text{proj}}, \partial_{y_i} A_{\text{proj}}, 0)$ is tangential to the substrate. The same area is obtained by closing the surface with a flat *base* triangulation T_{base} of the polygon and summing its signed triangle areas (the sign is taken with respect to the $-z$ orientation of the base). The tetrahedra spanned by base triangles and the origin, which lies in the plane of the base, have zero volume, so the base does not change the volume (29). The energy of the closed mesh is therefore a sum of per-triangle weights w_t times triangle area,

$$E = \sum_{t \in T_{\text{dome}}} w_t \, \text{Area}_t + \sum_{t \in T_{\text{base}}} \big(-\gamma_{lv} \cos\theta_Y \big) \, \text{Area}_t^{\pm}, \qquad w_t = \begin{cases} \gamma_{lv} & \text{free dome triangle,} \\ 0 & \text{wetted by the sphere,} \end{cases} \tag{36}$$

which equals Eq. (9) exactly, since $\sum_{T_{\text{base}}} \text{Area}_t^{\pm} = A_{\text{proj}}$ for any triangulation of the polygon, folded or not. The gradient of (36) is the area gradient (25)–(26) with each triangle's contribution multiplied by its weight.

Heterogeneous wall: polygon flux. Let the contact polygon be P and let $\mathbf{F}$ be the field of Eq. (15) with $\operatorname{div}\mathbf{F} = s$. By the divergence theorem, $\int_P s\,\mathrm{d}A = \oint_{\partial P} \mathbf{F}\cdot\mathbf{n}\,\mathrm{d}l$; on each straight edge $e = [p_a, p_b]$ the integrand is smooth (piecewise polynomial in the edge parameter divided by ρ), and

$$\int_P s\,\mathrm{d}A = \sum_e \int_e \mathbf{F}\cdot\mathbf{n}_e\,\mathrm{d}l \;\approx\; \sum_e \sum_{q=1}^{Q} w_q\,\mathbf{F}(\xi_q)\cdot\mathbf{n}_e\,|e|, \tag{37}$$

with Gauss–Legendre nodes ξ_q and weights w_q on the edge; the number of nodes Q must resolve the transition layer of s relative to the edge length. The exact first variation (17) is discretized with the same quadrature and the piecewise linear hat function φ_a of vertex a on the adjacent edges:

$$\frac{\partial E_{\mathrm{anom}}}{\partial \mathbf{p}_a} = -\gamma_{lv}(\kappa_a - \kappa_0)\sum_{e\ni a}\sum_q w_q\,s(\rho(\xi_q))\,\varphi_a(\xi_q)\,\mathbf{n}_e|e|. \tag{38}$$

Only contact-line vertices carry a gradient. For a polygon fully inside the disc ($s \equiv 1$) the quadrature reproduces $\partial A_{\mathrm{proj}}/\partial\mathbf{p}_a$, i.e. it is consistent with (35), and for a polygon not touching the disc both vanish.

Sphere wetting: solid angle by turning angles. For the loop of unit vectors $u_j = (p_j - \mathbf{c})/R$ of the wetted-patch boundary on the sphere, the enclosed solid angle follows from the discrete Gauss–Bonnet formula in terms of the turning angles τ_j (the signed angle in the tangent plane at u_j between the incoming and the outgoing edge of the loop),

$$\Omega = 2\pi - \sum_{j=1}^{n}\tau_j, \qquad A_{\mathrm{patch}} = R^2\,\Omega, \tag{39}$$

which is a discrete form of (19) for geodesic edges. The energy is

$$E_{\mathrm{wetting}} = -\gamma_{lv}\cos\theta_s\,R^2\Big(2\pi - \sum_j \tau_j\Big). \tag{40}$$

It is built from angle differences only, so it has no preferred direction on the sphere (no branch cut), at the price that "the enclosed side" is unambiguous only for a loop that is small compared with the sphere. A single τ_j depends only on the three vertices $j-1, j, j+1$, which is exploited when differentiating numerically (Section 8.1).

Force on the sphere, discrete. Differentiating the discrete energy with respect to $\mathbf{c}$ at fixed vertex positions, and accounting for the constraint that holds wetted vertices on the sphere, gives with $\mathbf{g}_i^{\mathrm{coll}}$ the collision gradient (Section 5.3), $\mathbf{g}_i^{\mathrm{Young}}$ the gradient of (40) with respect to $\mathbf{c}$ distributed over the loop vertices, $\mathbf{n}_i = (v_i - \mathbf{c})/|v_i - \mathbf{c}|$ and $\mathbf{g}_i^{E}$ the total gradient of the rest of the energy at vertex i:

$$\mathbf{F} = \sum_i \mathbf{g}_i^{\mathrm{coll}} + \sum_{i\in\mathrm{wet}} \mathbf{g}_i^{\mathrm{Young}} - \sum_{i\in\mathrm{wet}} (\mathbf{g}_i^{E}\cdot\mathbf{n}_i)\mathbf{n}_i. \tag{41}$$

The first term is the push of liquid that tries to penetrate the sphere, the second is the Young term of the patch boundary, the third is the radial part of the energy gradient on wetted vertices, which the constraint absorbs (liquid pressure on the patch plus the radial pull of the contact line). Eq. (41) needs no finite differences with respect to the sphere position.

4.8 Discrete Centre of Mass

Applying (20) with the divergence theorem to the closed triangulated surface, with signed tetrahedron volumes $V_t = \frac{1}{6}\, p_1 \cdot (p_2 \times p_3)$ with respect to the origin and the tetrahedron centroids $(p_1 + p_2 + p_3)/4$,

$$\mathbf{r}_{\mathrm{COM}} = \frac{\sum_t V_t\,(p_1 + p_2 + p_3)/4}{\sum_t V_t}. \tag{42}$$

Base triangles lie in the plane through the origin, so their tetrahedra vanish and the open dome and the closed mesh give the same value.

5 Constraints and the Optimisation Method

5.1 Exact Lagrange Projection

To keep the volume exactly conserved (to first order) at every gradient step, we project the area gradient $g_A = \nabla A$ onto the tangent space of the constraint surface $\{x : V(x) = V_0\}$. This tangent space is $T_x = \{h : \nabla V(x) \cdot h = 0\}$, and the orthogonal projection of g_A onto T_x is

$$g_{\mathrm{proj}} = g_A - \lambda\, g_V, \qquad \lambda = \frac{g_A \cdot g_V}{g_V \cdot g_V}, \tag{43}$$

where λ is chosen such that $g_V \cdot g_{\mathrm{proj}} = 0$ holds (i.e. the motion is volume-neutral to first order). This is the classical *gradient projection method* (Rosen), and it exactly matches the Lagrange multiplier condition (6) of the continuous problem: at the optimum, g_A is proportional to g_V and the projected gradient vanishes.

5.2 Penalty Formulation

The exact projection requires a Gram–Schmidt-like orthogonalization when several constraints are active simultaneously (multiple volumes, boundary conditions), which adds extra implementation cost. A simpler alternative, used by the program, is the *quadratic penalty method*: instead of (1), the *unconstrained* surrogate problem

$$\min_x \; E_k(x), \qquad E_k(x) := A(x) + k\,\big(V(x) - V_0\big)^2 \tag{44}$$

is solved for a fixed penalty weight $k > 0$, with gradient

$$\nabla E_k(x) = g_A(x) + 2k\big(V(x) - V_0\big)\, g_V(x). \tag{45}$$

It is a classical result of optimization theory that the minimizers of E_k converge to the minimizer of the constrained problem (1) as $k \to \infty$; for finite k, a small but generally non-vanishing volume deviation remains . The penalty method is particularly well suited in the context of a gradient flow (rather than an exact per-step minimization): it fits in seamlessly as an additional additive term in (45) and works unchanged for arbitrarily many simultaneous constraints (simply add more penalty terms).

Whenever further energies are present (wall, wetting), A in (44) is replaced by the total energy E and g_A in (45) by ∇E; this is the gradient g^E used in Part II.

5.3 Non-Penetration of a Rigid Sphere

A rigid sphere of radius R and (possibly time-dependent) centre $\mathbf{c}(t)$ must not be penetrated by the liquid. For a vertex v_i define the signed gap $g_i = \|v_i - \mathbf{c}\| - R$, negative inside the sphere.

Like the volume constraint, the constraint $g_i \geq 0$ is enforced softly by one-sided quadratic penalties,

$$E_{\text{sphere}} = \underbrace{k_c \sum_{i:\, g_i<0} g_i^2}_{\text{collision}} + \underbrace{k_a \sum_{i:\, 0\leq g_i<r_c} g_i^2}_{\text{adhesion (optional)}}, \qquad \nabla_{v_i} E_{\text{sphere}} = 2k\, g_i\, \hat{n}_i, \quad \hat{n}_i = \frac{v_i - \mathbf{c}}{\|v_i - \mathbf{c}\|}, \tag{46}$$

where r_c is a capture radius. Both pieces are continuous and vanish at $g_i = 0$. A substrate-penetration term of identical one-sided form, $k_s \sum_{i:\, z_i<0} z_i^2$, keeps the whole surface above the wall. These penalties are *not* derived from Young's equation; they only prevent interpenetration. Where a vertex is part of a genuine wetted patch, the exact energy (40) takes over its tangential behaviour and the radial component of the gradient is removed by a bilateral constraint (Section 8.1). Because the penalties are one-sided and local, the model keeps the topology of the surface: it can deform around the sphere, but it cannot pinch off into a detached bridge.

5.4 Gradient Flow and Explicit Euler Integration

The gradient flow $\partial x/\partial t = -\nabla E_k(x)$ is – as shown in Section 2 – at its core the mean curvature flow, augmented with the penalty term. The simplest numerical time integration is an explicit Euler step with step size η :

$$x^{n+1} = x^n - \eta\, \nabla E_k(x^n). \tag{47}$$

Mean curvature flow is a second-order parabolic equation; explicit time integration of it is subject to a CFL-like stability condition $\eta = O(h^2)$, where h is the typical edge length of the mesh [4] – as the mesh is refined, the step size must therefore shrink quadratically, which severely slows convergence on fine meshes.

5.5 Sobolev Preconditioning

This stability restriction arises because the L^2 gradient weights high-frequency (rough, oscillating between neighboring vertices) perturbations just as strongly as low-frequency (smooth, large-scale) perturbations – yet the curvature energy reacts far more sensitively to high-frequency perturbations. The idea of *Sobolev gradients* [5] (applied to geometric flows, among others, in [6]) is to form the gradient not with respect to the L^2 inner product, but with respect to an H^1-type inner product

$$\langle u, v\rangle_{\text{Sob}} := \langle u, v\rangle_{L^2} + \alpha\, \langle \nabla u, \nabla v\rangle_{L^2}. \tag{48}$$

Discretely, the (graph) Laplacian L (see below) is used as an approximation of $-\Delta$, and the Sobolev gradient g_{Sob} is implicitly defined by the requirement

$$\langle g_{\text{Sob}}, h\rangle_{\text{Sob}} = \langle g_{L^2}, h\rangle_{L^2} \quad \forall h, \tag{49}$$

which leads to the linear system

$$\boxed{(I + \alpha L)\, g_{\text{Sob}} = g_{L^2} \iff g_{\text{Sob}} = (I + \alpha L)^{-1} g_{L^2}} \tag{50}$$

The operator $(I + \alpha L)^{-1}$ is a low-pass filter: it strongly damps high-frequency components of g_{L^2} while leaving low-frequency ones nearly unchanged. In practice, this means the gradient step is "smoothed" before it is applied, which allows substantially larger stable step sizes η and increases the effective convergence speed (see the method comparison, where Sobolev gradient descent reaches lower energy than plain gradient descent at the same iteration count).

5.6 The Discrete Laplace Operator

We use the simple (unnormalized) graph Laplacian over the mesh edges: for vertices i, j with an edge (i, j) in the mesh,

$$L_{ij} = \begin{cases} \deg(i) & i = j \\ -1 & (i,j) \text{ is a mesh edge} \\ 0 & \text{otherwise,} \end{cases} \tag{51}$$

with $\deg(i)$ the number of neighbors of i. The linear system (50) is sparse and is solved componentwise (x, y, z separately); its matrix depends only on the mesh topology, so one factorization serves all iterations until the topology changes.

5.7 Trust-Region Step Capping

Sobolev smoothing controls the stability of the flow as a whole, but does not bound how far any INDIVIDUAL vertex can move in one step – which matters once the mesh has strongly non-uniform edge lengths (e.g. right after local refinement near the obstacle, Section 10): a short edge there can tolerate only a much smaller absolute displacement before the triangle degenerates than a long edge elsewhere. When a fraction f is set, the step (47) is rescaled per vertex so that no vertex moves further than f times the shortest edge currently incident to it (floored at a quarter of the mesh's median edge length, so one degenerate edge cannot freeze its vertex entirely):

$$x_i^{n+1} = x_i^n - \min\Big(1,\ \frac{f \cdot \ell_i}{\|\eta\, g_{\mathrm{Sob},i}\|}\Big)\, \eta\, g_{\mathrm{Sob},i}, \qquad \ell_i = \max\big(\min_{j\sim i}\|x_i - x_j\|,\ \tfrac{1}{4}\,\mathrm{median}(\ell)\big), \tag{52}$$

i.e. the ordinary Sobolev step (50) whenever it is already within the cap, uniformly shrunk towards it otherwise; the direction of $g_{\mathrm{Sob},i}$ is never altered, only its magnitude. This is a per-vertex, per-step safety bound layered on top of the smoothed gradient, not a replacement for the role of η in (47).

Part II
Implementation

6 Program Structure

Part II describes what the program `droplet_relaxer` does with the model of Part I. Statements about *numerical devices* are explicitly marked as such: they change the path of the iteration or the quality of the mesh, but are not derived from the energy.

The data flow is: `main.py` builds the mesh and scales the configuration; `solver.relax` is a Python generator that performs iterations and yields to the caller (which updates the window and decides on remeshing); when a remesh trigger fires, the segment ends, the mesh is changed, and a new `relax` segment continues at the same iteration number.

7 Meshes, Scaling and Degrees of Freedom

Start mesh. The default start mesh is a "disk": a cone-like dome of `n_rings` rings over a circle of unit radius, where ring k carries $6k$ vertices (hexagonal ring layout, giving nearly equilateral triangles), centred on an apex vertex and with the outermost ring on $z = 0$. A UV-sphere cap is available as alternative. The start height `start_height` fixes the initial volume.

Module	Content
`config.py`	All parameters (dataclasses `MeshConfig`, `SolverConfig`, `ObstacleConfig`, `VisConfig`, `AnomalyConfig`); the only file a user edits.
`main.py`	Start mesh, scaling to the target volume, remesh scheduling, live/off-screen loop, summary output.
`mesh.py`	Start meshes, DOF mask, global subdivision, Rivara refinement, edge collapse, tangential smoothing.
`substrate.py`	Closed mesh: base triangulation (Tutte embedding / ear clipping), per-triangle energy weights.
`energy.py`	Area, volume and their gradients, graph Laplacian, curvature for colouring, wall energy, substrate penetration, centre of mass.
`obstacle.py`	Straight sphere path $\mathbf{c}(u)$, collision/adhesion penalty.
`quasistatic.py`	Event-driven sphere stepping: convergence test, step advance, adaptive rollback, ETA.
`wetting.py`	Wetted-set detection, contact loop, sphere wetting energy.
`anomaly.py`	Circular wettability anomaly (polygon flux and exact gradient).
`solver.py`	The relaxation generator `relax`, force on the sphere, step control.
`visualize.py`	Window, charts, progress bar, history slider, anomaly disc, COM block.
`export.py`	Save button: arrays, configuration, README, video.

Table 1: Modules of the program.

Scaling. The mesh is scaled uniformly so that its volume equals `target_volume` (scale factor s). Quantities with the dimension of a length or of an energy per length are scaled with it: the volume penalty by s^{-4}, the snap distance, capture radius and substrate size by s, and the colour limits of the curvature by $1/s$. Sphere path points and anomaly lengths are *not* scaled; they are given in the physical units of the scaled mesh.

Degrees of freedom. The contact-line (ring) vertices keep $z = 0$: the z-component of their gradient is set to zero before the step and the coordinate is reset to 0 afterwards (`mesh.build_dof_mask`). Their x, y components are free. This is the discrete form of the mixed boundary condition of Section 2.3 and is the *boundary-discrete* condition referred to in Part I .

Closed mesh. By default (`base_mesh_enabled`) the dome is closed by a flat base. Base interior vertices are not degrees of freedom: every iteration they are placed by a harmonic (Tutte) embedding of the ring (uniform-weight graph Laplacian, sparse LU per topology), which is fold-free for a convex ring. For a strongly non-convex ring the program falls back to an ear-clipping triangulation without interior vertices; the signed area keeps the energy correct even for a folded base. The Sobolev Laplacian, the trust region, the smoothing and the wetting adjacency use dome triangles only. At a remesh the base is stripped, the open dome is remeshed and the base is rebuilt (Delaunay of a canonical circle, then the embedding). The closed formulation reproduces the open-mesh energy and gradient to $\approx 10^{-13}$ for $\theta_Y = 60°, 90°, 120°$ and costs about 30% additional run time.

Sphere path and quasi-static stepping. The centre moves on the straight line from `center_start` to `center_end`, parametrised by $u \in [0, 1]$. In the default quasi-static mode (u is a state of the solver, not a function of the iteration number) the sphere stands at u_k until

the liquid has relaxed; then a frame is recorded and $u_{k+1} = u_k + \Delta/\ell$ (Δ = `step`, ℓ the path length, the last step is clipped to the end). A step counts as converged when, at two consecutive tests (every `check_every` iterations, not before `min_iter_per_step`), $|E - E_{\mathrm{prev}}|/(\gamma V_0^{2/3}) < \varepsilon_E$, $\max_k |F_k - F_{k,\mathrm{prev}}|/(\gamma V_0^{1/3}) < \varepsilon_F$ and the largest vertex move per iteration in the window, relative to $V_0^{1/3}$, is below ε_x (the raw gradient criterion `grad_tol` also counts). At equilibrium the force is $F = -\mathrm{d}E^*/\mathrm{d}c$ (envelope argument), so the recorded pairs (E, F) are consistent. A step that does not converge within `max_iter_per_step` is either accepted and marked (fixed step), or, with `adaptive`, rolled back to the last converged state and retried with $\Delta/2$ (down to `step`·`min_step_factor`); a quickly converged step doubles it again (up to `step`). Warm starts are used throughout. The old iteration-driven mode (`quasi.enabled=False`) uses $u = t$, the iteration fraction.

8 Assembly of Energy and Force

In every iteration the solver assembles the total gradient g in the following order (`solver.relax`):

1. Sphere centre from the path; wetted vertex set and contact loop (if wetting is on).
2. Triangle weights w_t (36); area A and its gradient (weights, signed base area), volume V and g_V. A guard $0.5V_0 < V < 1.5V_0$ protects against corrupt states.
3. $g^E = g_A + 2k(V - V_0)\, g_V$ (45). Only if there is no base: the wall term (35).
4. The anomaly term (38) (contact-line vertices only), the substrate-penetration penalty.
5. Sphere collision (and adhesion) (46), with the force accumulation of (41).
6. Young wetting term (40), with its force contribution.
7. Removal of the radial component on wetted vertices (with its force contribution); z-gradient set to zero on the ring and base.
8. Force and diagnostics stored; convergence test $\|g\| <$ `grad_tol`.

Then follow the Sobolev solve, the step, and the numerical devices of Section 9.

8.1 Wetted Set and Contact Loop

A vertex is *wetted* if $\big|\|v_i - \mathbf{c}\| - R\big| < d_{\mathrm{snap}}$ (`wetting_snap_distance`); the set is restricted to the largest connected component in the mesh graph. A patch needs at least `wetting_min_patch_vertices` vertices. The set has no memory. Triangles with three wetted vertices get weight 0. The ordered boundary of the patch is the loop used in (40); loops that wrap more than about 115° half-width raise `SolverDiverged`, and configurations that are not a single simple loop (several patches, pinch points) are skipped for that iteration. Wetted vertices are projected back onto the sphere after every step and the patch interior is relaxed (Section 9).

The gradient of (40) is computed by central finite differences ($\pm\varepsilon$ in each coordinate). Since a turning angle depends only on three consecutive loop vertices, vertices at least 3 apart can be perturbed simultaneously; a greedy colouring (`wetting._stencil_colors`) turns the cost of the loop into a handful of vectorized evaluations.

Lesson on the snap distance. `wetting_snap_distance` is scaled with the mesh, and a value larger than one edge length re-captures the first vertex row onto the sphere in every iteration, so the contact line can neither recede nor reach the Young angle. With the default $d_{\mathrm{snap}} = 0.01$ the static sphere angle at $\theta_s = 120°$ is 119–120° (wall: $\approx 118°$); with the earlier, too large value it was only $\approx 100°$.

8.2 Anomaly

`anomaly.py` implements (37) and (38). The number of Gauss–Legendre nodes per edge is chosen from the ratio of the transition width $2h$ to the edge length, in steps of 16 between 16 and 96. The gradient agrees with finite differences to $\approx 10^{-9}$ and the energy with a brute-force area integral to $\approx 10^{-5}$ relative error. At start-up the program prints a note if the transition width is smaller than the contact-line edge length (2 versus 5.12 in the default configuration): the polygon then cannot resolve the profile, and the anomaly acts as a nearly sharp step. A test with a hydrophilic patch at the contact line shifts the centre of mass by $+8$ in x.

8.3 Force on the Sphere

Eq. (41) is evaluated inside every iteration, not by differences of energies. In a static check, $F_z = -58.9$ compared with -59.95 from $\mathrm{d}E_{\mathrm{tot}}/\mathrm{d}z$ of equilibrium states ($\approx 2\%$). The older estimate $-(\mathrm{d}E/\mathrm{d}s)(\mathrm{d}z/\mathrm{d}s)$ gives only the component along the motion and is polluted by the liquid's own relaxation (correlation ≈ 0.12 with the direct force); it is kept only as cross-check (`force_from_energy`, `force_parallel` in the saved data). The plotted force is a trailing mean over `force_average_window` recorded frames.

Lateral force is a numerical lag force. On a homogeneous substrate $F_x, F_y \to 0$ at equilibrium (Section 3.4). The lateral force during a moving run is a pseudo-dynamic lag force of the gradient-flow relaxation; its size depends on the learning rate, Sobolev α, the trust region and the sphere speed, and vanishes for speed $\to 0$. It contains no viscous friction and no contact-angle hysteresis and should be used only comparatively at fixed settings. The sphere speed does not limit the contact line: halving the speed changes the radii by $< 1\%$.

9 The Iteration: Step and Numerical Devices

Algorithm 1 One relaxation segment (between two remeshings)

$V_0 \leftarrow$ target volume (from the start mesh); build L; factorize $M = I + \alpha L$ once per topology
for $n = n_0, n_0 + 1, \ldots$ **do**
  sphere centre $\mathbf{c}(n)$; wetted set and loop
  assemble g as in Section 8; store force
  if $\|g\| <$ tol **then stop**
  end if
  solve $M\, g_{\mathrm{Sob}} = g$ componentwise ▷ Eq. (50)
  $\delta x \leftarrow -\eta\, g_{\mathrm{Sob}}$ capped per vertex ▷ Eq. (52)
  $x \leftarrow x + \delta x$; ring $z \leftarrow 0$
  every N iterations: tangential smoothing
  re-embed the base; relax the interior of the wetted patch and project it onto the sphere
  finiteness and explosion guards; yield to caller
end for

Numerical devices (not derived from the energy; they change the path or mesh quality, not the minimizer of the model):

- **Sobolev solve.** `scipy.sparse.linalg.factorized` of $I + \alpha L$ (`energy.build_laplacian`); factorized again only after a topology change; applied to x, y, z separately. Default $\alpha = 3$.
- **Trust region.** Eq. (52) with $f =$ `max_step_fraction` $= 0.3$; the floor is $\frac{1}{4}$ of the median edge length.

- **Tangential smoothing.** Every `tangential_smoothing_every` = 10 iterations a fraction 0.3 of the offset to the neighbour centroid, with its normal component removed, is applied; also right after each remesh. Its purpose is mesh quality (triangles do not crowd at the apex); it does not follow the energy.
- **Patch interior.** Interior vertices of the wetted patch are relaxed and projected onto the sphere each iteration.
- **Guards.** Volume window, finiteness, step explosion, and the wrapping guard (`SolverDiverged`).
- **Soft penalties.** Volume, collision and substrate stiffness have absolute values; with $\gamma_{lv} = 0.88$ the surface forces are 12% weaker relative to them. The volume drifts by 2–3% from V_0 during contact.

Model limitations (measured). The wetted vertices are held by a *bilateral* constraint and detach only through remeshing. A wall-ring hysteresis of about 4% remains at $\theta_Y = 120°$; it is independent of the speed and its cause is not isolated. The relaxation is a pseudo-time process without inertia, viscosity or contact-line friction.

10 Adaptive Remeshing

A fixed mesh eventually becomes a poor approximation of the surface: regions stretched by the sphere obstacle develop long, thin triangles (a poor discretization of curvature there), while regions no longer under active deformation carry more resolution than they need. Unlike the energy terms and the gradient step, remeshing is not part of the variational problem itself – it never changes x by following a gradient, only the DISCRETIZATION (vertices/triangles) representing the same physical surface – so this section is purely about `mesh.py`'s operations and `main.py`'s scheduling of them, not about deriving new energy terms.

10.1 Two Refinement Modes

`MeshConfig.remesh_split_mode` selects between two ways of adding resolution, checked periodically (every `remesh_check_every` iterations) against the current longest edge relative to the mesh's original mean edge length (`remesh_trigger_ratio`):

- **Global (`subdivide_mesh`):** every triangle is split into 4 by inserting a vertex at each edge's midpoint. Because every edge gets exactly one midpoint, shared by both triangles bordering it, the result is always a valid mesh with no "T-junction" (a mismatched edge where one side is subdivided and its neighbor is not) – at the cost of refining the WHOLE mesh uniformly, including regions far from the obstacle that did not need it.

- **Local (`refine_rivara`):** Rivara longest-edge bisection [7], which refines only where an edge actually exceeds its target length – and, near the obstacle (`remesh_contact_len_ratio`), targets a deliberately finer length than elsewhere. The classical algorithm avoids T-junctions without touching the rest of the mesh: a triangle is bisected at its longest edge only once the triangle across that edge agrees that this IS its longest edge too (otherwise that neighbor is refined first, recursively – the propagation only ever walks towards longer edges, so it terminates).

Both preserve the boundary ring's cyclic structure and re-project any new boundary vertex exactly to $z = 0$ (guarding against floating-point drift, since its two parent vertices are already there by construction).

10.2 Edge Collapse

Repeated splitting grows the triangle count without bound, so `collapse_short_edges` prunes it back: whenever an edge (never one touching the boundary ring) falls below a length threshold, its two endpoints are merged to their midpoint, subject to two safety checks applied *before* committing:

- the *link condition* – the two endpoints' neighbor sets may intersect only in the (at most two) apex vertices of the triangles already adjacent to that edge; otherwise the collapse would pinch the surface into a non-manifold shape, and is skipped;
- no triangle that survives the collapse may flip its normal direction or degenerate (checked by comparing each affected triangle's normal before and after the trial move); otherwise the collapse is skipped.

A single pass never lets a committed collapse's neighborhood participate in a second collapse in the same pass (every vertex it touches is "frozen" for the rest of that pass), so `main.py` calls it repeatedly, once per remesh check, for further pruning. Local refinement's collapse counterpart additionally accepts a `protect_center`/`protect_radius` (the obstacle's current position): edges near it are excluded from the ordinary length threshold entirely, optionally down to a separate, much smaller `protect_min_len` that still lets a pathologically short edge there collapse (e.g. the wetted patch's contact line getting pinched by competing forces as it moves) without eroding the extra resolution the obstacle still needs. This asymmetric split-everywhere / collapse-outside-a-protected-region combination is what approximates genuinely local/adaptive remeshing without a fully general adaptive scheme: resolution still spreads globally on a split, but persists selectively near the obstacle once collapse starts pruning it back elsewhere.

10.3 Triggers and Mesh-Quality Smoothing

`main.py` checks (every `remesh_check_every` iterations) whether the longest edge exceeds `remesh_trigger_ratio` times the original mean edge length (triggering a split), or whether the FRACTION of all edges shorter than `remesh_min_edge_ratio` times that same reference exceeds `remesh_min_edge_fraction` (triggering a collapse pass) – a fraction, not the single minimum edge, because an "umbrella" mesh converging on one apex vertex structurally always has a handful of persistently short spoke edges there regardless of any obstacle, which would otherwise trigger collapse permanently. In local mode, two additional obstacle-local triggers (`long_edges_near`/`short_edges_near`) catch distortion concentrated near the obstacle before it shows up in either global metric. Whenever any trigger fires, the current segment of `solver.py:relax` (a Python generator) is stopped, the mesh is split or collapsed, and a fresh `relax` call resumes from the same iteration with the new vertex/triangle arrays – the sparse Laplacian factorization (50) (which depends on the mesh's edge topology) is otherwise only ever rebuilt at these boundaries, not on a fixed schedule. Independently of remeshing, `tangential_smooth_step` (used both right after a remesh event and periodically during relaxation, every `tangential_smoothing_every` iterations) moves every non-fixed vertex partway toward its neighbors' centroid with the along-normal component of that offset removed first – a pure mesh-quality cleanup (it does not follow a gradient of the area/volume/wetting energy and does not measurably change area, volume, or curvature) that keeps triangles from crowding together at high-curvature regions like the apex, which would otherwise shrink the trust-region step cap there (Section 5.7) and stall convergence.

Implementation: `mesh.py:subdivide_mesh`, `mesh.py:refine_rivara`, `mesh.py:collapse_short_edges`, `mesh.py:tangential_smooth_step`; scheduling in `main.py`.

10.4 Cost and Speed-ups

On the earlier default run (2500 iterations, no window) of about 25 s, the relaxation took about 63% and remeshing about 35% of the time. Two changes reduced it to 19–21 s with bit-identical results (area, volume, radius and 41 remeshings unchanged): `collapse_short_edges` was rewritten with a CSR adjacency in `numpy` and lazily built per-vertex neighbour sets (identical output on 215 recorded real calls; local 3.5 → 0.6 s, global 1.9 → 0.6 s), and edge extraction uses integer keys (`wetting.unique_edges`, 3–4× faster). Skipping the wetting step gave no gain (contact in about 85% of the iterations, patch about 450 of 2200 vertices); compiling the kernels with Numba made them 9× faster but the whole run only 10–15%, so it is not enabled.

11 Output and Visualization

Window. The surface is coloured by the unsigned curvature of Section 4.6; the ring is a cyan tube, the contact line on the sphere is lime. A wettability anomaly is drawn as a disc on the substrate (blue hydrophilic, violet hydrophobic). A three-line block at the lower left shows the centre of mass, its xy-offset to the sphere centre and the volume. The mesh is rendered every `render_every` iterations (default 200). Outlines are added with `render=False`, which halved the number of renders (887 → 419; 120 → 75 s with software OpenGL).

Charts and progress. Two charts, axis colours matching the axis tripod (x tomato, y green, z blue), both over the tip x-coordinate (sphere centre x) with fixed limits from the planned path, one point per converged step:

- *Left:* total surface free energy (9) (plus the anomaly, roughness and sphere patch terms).
- *Right:* F_x, F_y, F_z in the tripod colours (equilibrium values, no averaging).

Window and charts are only updated when a step has converged; in between a progress bar (fraction of the path, elapsed time, ETA from the wall time per path length of the recent steps) is refreshed about once per second. After the run the history slider runs over the tip x-coordinate. Diamond markers show the current frame.

Save. The Save button writes iteration, gap, energy, `frames.csv`, COM offset (N and $N \times 2$), sphere centre, force (averaged and raw), the cross-check forces, the used `config.py`, `config_effective.json`, a README and `run.mp4` (imageio, falling back to GIF; up to `video_max_frames` = 600 frames).

12 Verification

The following checks were carried out; none is an independent third-party validation.

- Closed versus open mesh: energy and gradient agree to $\approx 10^{-13}$ (60°, 90°, 120°).
- Contact angles at 120°: sphere 119–120°, wall $\approx$ 118°; cone COM height 10.72 versus $h/4 = 10.76$.
- Anomaly: gradient versus finite differences $\approx 10^{-9}$; energy versus brute-force integral $\approx 10^{-5}$.
- Force on the sphere versus $\mathrm{d}E/\mathrm{d}z$ of equilibrium states: $\approx 2\%$.
- Repeated runs reproduce area, volume and radius to all printed digits (the program contains no randomness).
- The off-screen pipeline is tested for both `remesh_split_mode` values with warnings treated as errors.

13 Mapping from Equations to Code

Equation / section	File : function	Meaning
(22), (25)–(26), (36)	`energy.py:area_and_grad`	Weighted area and gradient
(29), (31)	`energy.py:volume_and_grad`	Volume and gradient
(32)	`energy.py:mean_curvature`	Curvature for colouring
(44), (45)	`solver.py:relax`	Penalty gradient
Sec. 7	`mesh.py:build_dof_mask`	Contact-line DOFs
(34), (35)	`energy.py:substrate_wall_energy_and_grad`; `substrate.py`	Wall energy / base
(37), (38)	`anomaly.py`	Wettability anomaly
(46)	`obstacle.py`	Collision / adhesion
(39), (40)	`wetting.py:sphere_wetting_energy_and_grad`	Sphere wetting
(41)	`solver.py:relax`	Force on the sphere
(42)	`energy.py:center_of_mass`	Centre of mass
(50), (47)	`energy.py:build_laplacian`; `solver.py`	Sobolev step
(52)	`solver.py:relax`	Trust region
Sec. 10	`mesh.py`; `main.py`	Remeshing and scheduling

Table 2: Mapping of the equations to the code.

Part III
Comparison and Outlook

14 Comparison to Ken Brakke's Surface Evolver

`droplet_relaxer` is deliberately built on the same core mathematical machinery as Brakke's Surface Evolver [2] – discretize the surface as a triangle mesh, move vertices along an energy gradient – and Section 4.6 through Section 10 of this document derive that machinery from scratch. This section makes the relationship explicit: where the two agree in principle, and where this project is a narrower, special-purpose reimplementation rather than a general tool.

14.1 Where the Two Agree

The core discrete quantities are the same construction: Evolver's area and volume gradients, and its handling of a fixed-height substrate constraint via a restricted degree-of-freedom space, are the same discrete differential-geometry identities derived independently in Sections 4.3–4.6 and 7 of this document. Both tools are, at bottom, performing a gradient flow of the same functional (1) on a piecewise-linear surface.

14.2 Where This Project Is Narrower

Generality. Evolver is a general-purpose, scriptable tool: an interactive command language for describing essentially arbitrary variational surface problems (multiple bodies, multiple volume/pressure constraints, arbitrary constraint surfaces, user-defined energy quantities) and driving many different evolution strategies over them. `droplet_relaxer` is a fixed single-purpose Python program for exactly one geometry: a single droplet, a single flat substrate, and at most one moving spherical obstacle (`config.py`); every energy term

in Sections 4.6–8.1 is hardcoded into `energy.py`/`wetting.py`/`obstacle.py` rather than expressed in a general constraint/quantity language. Extending this project to a genuinely new geometry means writing new Python, not a new script.

Constraint handling. Evolver's volume constraints are handled by (an efficient, generalized form of) the exact Lagrange projection of Section 5 (43), applied exactly at every step. `droplet_relaxer` instead uses the simpler quadratic penalty (44), which only converges to exact volume conservation as the penalty weight $k \to \infty$ and in practice leaves a small residual $|V - V_0|$ at any finite k and iteration count (Section 5 discusses why the penalty method was chosen here anyway: it needs no orthogonalization against several simultaneous constraints and composes as a plain additive gradient term).

Solver method. Evolver's default optimization step is plain (unpreconditioned) L^2 gradient descent, with an optional conjugate-gradient and Hessian (Newton-type) mode available as alternatives. `droplet_relaxer` implements only the Sobolev-preconditioned gradient descent of Section 5.5 – there is no Newton/Hessian mode in this program. A separate method-comparison experiment in this project (outside the scope of this document; see the project materials) found that for this specific problem, Newton-type steps – even with an exact Hessian-vector product via automatic differentiation – underperform the Sobolev gradient here, plausibly due to nearly-degenerate tangential degrees of freedom in the mesh (moving a vertex along the surface, rather than across it, barely changes the energy, which makes the Hessian ill-conditioned in those directions); this is specific to this experiment's setup, not a general claim about Newton methods on this class of problem.

Mesh topology and remeshing. Evolver's simplicial complex supports genuine topology changes (Brakke1992) – edge flips (equiangulation), and vertex/edge/facet insertion and deletion general enough to let a thin neck pinch off into separate bodies. `droplet_relaxer`'s remeshing (Section 10) only ever changes *resolution* on a single, fixed-topology open surface: global 1-to-4 subdivision, local Rivara refinement, and a conservative, link-condition-gated edge collapse – no edge flips, and no operation that could create or remove a boundary component. A droplet being dragged by the sphere obstacle can therefore stretch into an arbitrarily thin neck, but never pinches off into a detached liquid bridge (Section 5.3).

Contact energies. Evolver's general constraint framework can, in principle, express an exact contact-angle energy against an arbitrary constraint surface (not just a plane). This project implements only two contact energies, each specific to its own geometry: the exact line-integral Young's-equation term against the flat substrate (Sections 3.1 and 7, via the planar shoelace formula (34) or, equivalently, the base area) and, separately, the exact discrete-Gauss-Bonnet spherical version against the sphere obstacle (Section 8.1). Both are exact for their one specific surface (plane, sphere), constructed by different means (a planar vector potential vs. a gauge-free solid-angle sum) – neither generalizes automatically to an arbitrary obstacle shape the way Evolver's constraint mechanism is designed to.

Maturity and validation. Evolver is production-grade software with a three-decade publication record across many independent research groups. `droplet_relaxer` is a from-scratch reimplementation of the same core mathematics, written and tested for this one project; its correctness rests on the gradient derivations in this document and this project's own tests (e.g. the checks listed in Section 12), not on independent third-party validation.

15 Context and Outlook

Possible extensions. Ordered roughly by their physical significance; the first group removes limitations stated in Part II, the second adds physics, the third improves numerics.

Physics and model

- **Topology change / pinch-off** (still open, the most important): track the contact loop on the sphere and re-triangulate so that a thin neck can break into a detached bridge (Section 14). Today the topology is fixed (Section 10).
- **Contact-angle hysteresis, pinning and contact-line dissipation:** replace the pseudo-dynamic lag force (Section 3.4) by a physical law, e.g. advancing/receding angles and a viscous contact-line friction, so that lateral forces and stick–slip over the anomaly become quantitative. This also needs a physical time scale instead of the learning rate.
- **General heterogeneous walls:** several discs, stripes, or an arbitrary field $\kappa(x, y)$. The flux construction of Section 3.2 works for any field with a computable primitive; a general field would need a numerical primitive on a background grid.
- **Unilateral wetting constraint:** allow wetted vertices to leave the sphere without remeshing (a contact inequality instead of the bilateral projection), so that the contact line can recede smoothly.
- **General, rotating or multiple obstacles,** including torque on the obstacle. The solid-angle construction is specific to a sphere; other shapes need a different area functional on the obstacle surface.
- **Sheets wrapping around the sphere:** multi-chart or volume/phase-field representations remove the limit on the wetted half-width.
- **Gravity:** the term $\int \rho g z \, \mathrm{d}V$, with the gradient from the divergence theorem as for the volume (Section 4.3).
- **Multiple volumes / phases:** the penalty sum (44) and the projection (43) generalize to several constraints.

Constraints and accuracy

- **Exact volume conservation:** an augmented Lagrangian or exact projection instead of the pure penalty, to remove the 2–3% volume drift during contact.
- **Analytic gradient of the sphere wetting energy** instead of finite differences with stencil colouring.
- **Validation:** comparison with axisymmetric Young–Laplace capillary-bridge solutions and Surface Evolver benchmarks, and a mesh-convergence study; isolate the remaining $\approx 4\%$ wall hysteresis at 120°.

Numerics

- **Faster and adaptive stepping:** line search, L-BFGS or an adaptive step instead of the fixed η; compiled kernels or cached topology data for the hot loops (the Numba test gave only 10–15%, so the Python-level overhead and the remeshing dominate).
- **Mesh quality:** edge flips (equiangulation), cotangent-weighted or Lloyd-type tangential regularization instead of the centroid smoothing, and a fully adaptive local collapse with a spatially varying target length (for several obstacles).

References

[1] K. A. Brakke, *The Motion of a Surface by Its Mean Curvature*, Mathematical Notes 20, Princeton University Press, 1978.

[2] K. A. Brakke, *The Surface Evolver*, Experimental Mathematics **1**(2), 141–165, 1992.

[3] M. Meyer, M. Desbrun, P. Schröder, A. H. Barr, *Discrete Differential-Geometry Operators for Triangulated 2-Manifolds*, in: Visualization and Mathematics III, Springer, 2003.

[4] M. Desbrun, M. Meyer, P. Schröder, A. H. Barr, *Implicit Fairing of Irregular Meshes using Diffusion and Curvature Flow*, Proc. SIGGRAPH 1999.

[5] J. W. Neuberger, *Sobolev Gradients and Differential Equations*, Lecture Notes in Mathematics 1670, Springer, 1997.

[6] G. Sundaramoorthi, A. Yezzi, A. C. Mennucci, *Sobolev Active Contours*, International Journal of Computer Vision **73**(3), 345–366, 2007.

[7] M.-C. Rivara, *Algorithms for Refining Triangular Grids Suitable for Adaptive and Multigrid Techniques*, SIAM Journal on Numerical Analysis **21**(3), 604–613, 1984.

[8] W. T. Tutte, *How to Draw a Graph*, Proceedings of the London Mathematical Society **s3-13**, 743–767, 1963.